

Project Title: AI-Based Real-Time Voice-to-Voice Language Translator

Index

SR.NO	CONTENT	PAGE NO.
1	Abstract	1
2	Introduction	2
3	Methodology	3-4
4	Code	5-10
5	Output	11-12
6	Feature of the Project	13
7	Conclusion	14
8	References	15

Abstract

This project introduces a real-time voice-to-voice language translator using speech recognition, machine translation, and text-to-speech (TTS). Built with Python and GUI using Tkinter, it provides support for multiple languages, audio output, and MP3 saving. In a multilingual world, effective communication across different languages remains a significant challenge. This project introduces an AI-based real-time voice-to-voice language translator designed to overcome language barriers by combining multiple intelligent technologies. The system utilizes speech recognition to convert spoken input into text, automatically detects the input language, translates it into a selected target language using machine translation, and then synthesizes the translated text into speech using text-to-speech (TTS) capabilities. Developed using Python, the application integrates libraries such as speech recognition, googletrans, gTTS, and pygame, along with a graphical user interface built with Tkinter for ease of use.

Introduction

Language is one of the biggest barriers to global communication. With the rise of international travel, remote work, and global collaboration, the demand for multilingual tools has increased significantly. Traditional translation methods like dictionaries and manual input are time-consuming and impractical for real-time conversations. This project presents a real-time voice-to-voice translator that uses Artificial Intelligence to detect, translate, and speak human language. It combines speech recognition, language detection, machine translation, and text-to-speech (TTS) synthesis to create a seamless communication experience. The system is built using Python and integrates various libraries such as recognition, googletrans, gTTS, and pygame. It features a user-friendly GUI using Tkinter that allows users to interact easily. Users can select output languages, start or stop recording, and save MP3 files of the translated speech.

Methodology

The development of the AI-based real-time voice-to-voice language translator followed a structured and iterative methodology. The key phases include requirement analysis, system design, implementation, testing, and evaluation

1. Requirement Analysis

- Identified the primary objective: enabling real-time voice translation between multiple languages.
- Conducted a survey on common communication barriers faced by users in multilingual contexts.
- Determined the need for automatic speech recognition, language detection, machine translation, and speech synthesis.
- Selected supported languages based on global usage and technical feasibility.
- Established user requirements such as real-time response, GUI simplicity, and offline usability options.

2. System Design

- Designed a modular architecture consisting of input (speech), processing (translation), and output (audio).
- Chose Python as the core development language for its strong support in AI and language libraries.
- Created a graphical user interface using Tkinter for ease of interaction.
- Defined the interaction between components using class-based encapsulation in Python.
- Planned the use of threads to ensure GUI responsiveness during continuous listening

3. Technology Stack Selection

- speech recognition for real-time voice input.
- googletrans for language detection and translation.
- gTTS (Google Text-to-Speech) for converting translated text into audio. • pygame for audio playback.
- Tkinter for GUI development

4. Implementation

- Developed a class named VoiceTranslatorApp encapsulating all functionalities.
- Integrated microphone access with speech recognition and exception handling.
- Implemented auto language detection with fallback to English.
- Handled translation of recognized text using Google Translate API. • Used gTTS to convert translated text into an audio file and play it.
- Allowed users to save the translated output as an MP3 file

5. Testing and Debugging

- Performed unit testing for individual functions (e.g., speech recognition, translation).
- Conducted integration testing across the complete voice-to-voice pipeline.
- Simulated real-time usage scenarios to identify latency and response issues.
- Handled exceptions for audio failures, API limits, and language code mismatches.

Code

```
import speech_recognition as sr
from googletrans import Translator
from gtts import gTTS
import pygame
import threading
import tkinter as tk
from tkinter import ttk, messagebox, simpledialog
import uuid
import os
import time
from fpdf import FPDF # <- NEW IMPORT

# for Output translation
LANGUAGES = {
    'en': 'English',
    'es': 'Spanish',
    'fr': 'French',
    'de': 'German',
    'hi': 'Hindi',
    'ar': 'Arabic',
    'zh-cn': 'Chinese (Simplified)',
    'ja': 'Japanese',
    'ru': 'Russian',
    'it': 'Italian'
}

TTS_LANG_CODES = {
    'en': 'en',
    'es': 'es',
    'fr': 'fr',
    'de': 'de',
    'hi': 'hi',
    'ar': 'ar',
    'zh-cn': 'zh-cn',
    'ja': 'ja',
    'ru': 'ru',
    'it': 'it'
}

class VoiceTranslatorApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Auto Language Detection Voice Translator")
        self.root.geometry("620x650")
        self.root.configure(bg="#282c34")
        self.root.resizable(False, False)
```

```

pygame.mixer.init()
self.recognizer = sr.Recognizer()
self.translator = Translator()
self.listening = False
self.last_translated_text = ""

self.create_widgets()

def create_widgets(self):
    label_style = {'font': ("Helvetica", 11, "bold"), 'bg': '#282c34', 'fg': 'white'}
    text_style = {'font': ("Helvetica", 11), 'bg': '#3c3f41', 'fg': 'white', 'insertbackground': 'white'}

    frame = tk.Frame(self.root, bg="#282c34", padx=20, pady=20)
    frame.pack(fill="both", expand=True)

    self.auto_detect_var = tk.BooleanVar(value=True)
    auto_chk = tk.Checkbutton(frame, text="Auto Detect Input Language",
                             variable=self.auto_detect_var,
                             bg="#282c34", fg="white", font=("Helvetica", 11, "bold"),
                             activebackground="#282c34", activeforeground="white", selectcolor="#282c34")
    auto_chk.grid(row=0, column=0, sticky="w", pady=(0, 10))

    tk.Label(frame, text="Output Language (Translation):", **label_style).grid(row=1,
                                     column=0, sticky="w")
    self.output_lang_var = tk.StringVar(value='en')
    output_dropdown = ttk.Combobox(
        frame, textvariable=self.output_lang_var, values=list(LANGUAGES.keys()),
        state='readonly',
        font=("Helvetica", 11))
    output_dropdown.grid(row=2, column=0, sticky="ew", pady=5)
    output_dropdown.bind("<<ComboboxSelected>>", self.update_output_lang_label)
    self.output_lang_label = tk.Label(frame, text=f'({LANGUAGES["en"]})',
                                     **label_style)
    self.output_lang_label.grid(row=2, column=1, sticky="w")

    tk.Label(frame, text="Recognized Text:", **label_style).grid(row=3, column=0,
                                                                  sticky="w", pady=(15, 0))
    self.recognized_text = tk.Text(frame, height=6, width=70, wrap="word", **text_style,
                                   state='disabled', relief="solid", borderwidth=1)
    self.recognized_text.grid(row=4, column=0, columnspan=2, pady=(0, 10))
    recognized_scroll = ttk.Scrollbar(frame, command=self.recognized_text.yview)
    recognized_scroll.grid(row=4, column=2, sticky='ns', pady=(0, 10))
    self.recognized_text['yscrollcommand'] = recognized_scroll.set

    tk.Label(frame, text="Translated Text:", **label_style).grid(row=5, column=0,
                                                                  sticky="w", pady=(10, 0))
    self.translated_text = tk.Text(frame, height=6, width=70, wrap="word", **text_style,
                                   state='disabled', relief="solid", borderwidth=1)
    self.translated_text.grid(row=6, column=0, columnspan=2)

```



```
translated_scroll = ttk.Scrollbar(frame, command=self.translated_text.yview)
translated_scroll.grid(row=6, column=2, sticky='ns')
self.translated_text['yscrollcommand'] = translated_scroll.set
```

```
btn_frame = tk.Frame(frame, bg="#282c34")
btn_frame.grid(row=7, column=0, columnspan=3, pady=20, sticky="ew")
```

```
self.start_btn = tk.Button(btn_frame, text="Start Listening",
command=self.start_listening,
bg="#4caf50", fg="white", font=("Helvetica", 11, "bold"),
relief="flat", padx=15, pady=8)
self.start_btn.pack(side="left", padx=5, fill="x", expand=True)
```

```
self.stop_btn = tk.Button(btn_frame, text="Stop Listening",
command=self.stop_listening,
bg="#f44336", fg="white", font=("Helvetica", 11, "bold"),
relief="flat", padx=15, pady=8, state="disabled")
self.stop_btn.pack(side="left", padx=5, fill="x", expand=True)
```

```
self.save_btn = tk.Button(btn_frame, text="Save Translation as MP3",
command=self.save_translation_mp3,
bg="#2196f3", fg="white", font=("Helvetica", 11, "bold"),
relief="flat", padx=15, pady=8)
self.save_btn.pack(side="left", padx=5, fill="x", expand=True)
```

```
# NEW BUTTON: Save as PDF
self.save_pdf_btn = tk.Button(btn_frame, text="Save as PDF",
command=self.save_translation_pdf,
bg="#9c27b0", fg="white", font=("Helvetica", 11, "bold"),
relief="flat", padx=15, pady=8)
self.save_pdf_btn.pack(side="left", padx=5, fill="x", expand=True)
```

```
def update_output_lang_label(self, event=None):
lang_code = self.output_lang_var.get()
lang_name = LANGUAGES.get(lang_code, "")
self.output_lang_label.config(text=f'({lang_name})')
```

```
def start_listening(self):
if self.listening:
return
self.listening = True
self.start_btn.config(state='disabled')
self.stop_btn.config(state='normal')
```

```
threading.Thread(target=self.listen_and_translate_loop, daemon=True).start()
```

```
def stop_listening(self):
self.listening = False
self.start_btn.config(state='normal')
self.stop_btn.config(state='disabled')
```

```

def listen_and_translate_loop(self):
    try:
        with sr.Microphone() as source:
            self.recognizer.adjust_for_ambient_noise(source)

    while self.listening:
        audio = self.recognizer.listen(source, phrase_time_limit=5)

        try:
            recognized_text = self.recognizer.recognize_google(audio, language='en-US')

            if self.auto_detect_var.get():
                detected = self.translator.detect(recognized_text)
                input_lang = detected.lang
            else:
                input_lang = 'en'

            self.append_text(self.recognized_text, f"[{input_lang}] " + recognized_text)

            output_lang = self.output_lang_var.get()
            translated = self.translator.translate(recognized_text, src=input_lang,
            dest=output_lang)
            self.append_text(self.translated_text, translated.text)

            self.last_translated_text = translated.text
            self.speak_text(translated.text, output_lang)

        except sr.UnknownValueError:
            self.append_text(self.recognized_text, "[Could not understand audio]")
        except sr.RequestError as e:
            self.append_text(self.recognized_text, f"[Speech service error: {e}]")
        except Exception as e:
            self.append_text(self.recognized_text, f"[Error: {e}]")

    except Exception as e:
        messagebox.showerror("Error", f"Microphone error:\n{e}")
        self.listening = False
        self.start_btn.config(state='normal')
        self.stop_btn.config(state='disabled')

    def append_text(self, widget, text):
        widget.config(state='normal')
        widget.insert('end', text + "\n")
        widget.see('end')
        widget.config(state='disabled')

    def speak_text(self, text, lang_code):
        lang = TTS_LANG_CODES.get(lang_code, 'en')
        try:

```

```

filename = f'temp_{uuid.uuid4().hex}.mp3"
tts = gTTS(text=text, lang=lang)
tts.save(filename)
pygame.mixer.music.load(filename)
pygame.mixer.music.play()
while pygame.mixer.music.get_busy():
    time.sleep(0.1)
os.remove(filename)
except Exception as e:
    self.append_text(self.recognized_text, f"[TTS Error: {e}]")

def save_translation_mp3(self):
    if not self.last_translated_text.strip():
        messagebox.showwarning("Warning", "No translated text to save!")
    return

filename = simplifiedialog.askstring("Save MP3", "Enter file name (without
extension):")
if not filename:
    return

filename = filename.strip()
if not filename:
    messagebox.showwarning("Warning", "Invalid file name!")
    return

lang = TTS_LANG_CODES.get(self.output_lang_var.get(), 'en')
try:
    tts = gTTS(text=self.last_translated_text, lang=lang)
    full_path = filename + ".mp3"
    tts.save(full_path)
    messagebox.showinfo("Success", f"File saved successfully:\n{full_path}")
except Exception as e:
    messagebox.showerror("Error", f"Failed to save MP3:\n{e}")

# NEW FUNCTION
def save_translation_pdf(self):
    if not self.last_translated_text.strip():
        messagebox.showwarning("Warning", "No translated text to save!")
    return

filename = simplifiedialog.askstring("Save PDF", "Enter file name (without
extension):")
if not filename:
    return

filename = filename.strip()
if not filename:
    messagebox.showwarning("Warning", "Invalid file name!")
    return

```

```
try:
pdf = FPDF()
pdf.add_page()
pdf.set_auto_page_break(auto=True, margin=15)
pdf.set_font("Arial", size=12)

pdf.set_font("Arial", 'B', 14)
pdf.cell(200, 10, txt="Voice Translator Output", ln=True, align='C')
pdf.ln(10)

pdf.set_font("Arial", 'B', 12)
pdf.cell(200, 10, txt="Recognized Text:", ln=True)
pdf.set_font("Arial", size=12)
original_text = self.recognized_text.get("1.0", "end").strip()
pdf.multi_cell(0, 10, original_text)
pdf.ln(5)

pdf.set_font("Arial", 'B', 12)
pdf.cell(200, 10, txt="Translated Text:", ln=True)
pdf.set_font("Arial", size=12)
pdf.multi_cell(0, 10, self.last_translated_text)

full_path = filename + ".pdf"
pdf.output(full_path)
messagebox.showinfo("Success", f"PDF saved successfully:\n{full_path}")
except Exception as e:
messagebox.showerror("Error", f"Failed to save PDF:\n{e}")

if __name__ == "__main__":
root = tk.Tk()
app = VoiceTranslatorApp(root)
root.mainloop()
```

Output

Auto Language Detection Voice Translator

☒ Auto Detect Input Language

Output Language (Translation):

hi

 (Hindi)

Recognized Text:

Translated Text:

Start Listening

Stop Listening

Save Translation as MP3

Save

Auto Language Detection Voice Translator

☒ Auto Detect Input Language

Output Language (Translation):

hi

 (Hindi)

Recognized Text:

[en] hello my name is Omkar

Translated Text:

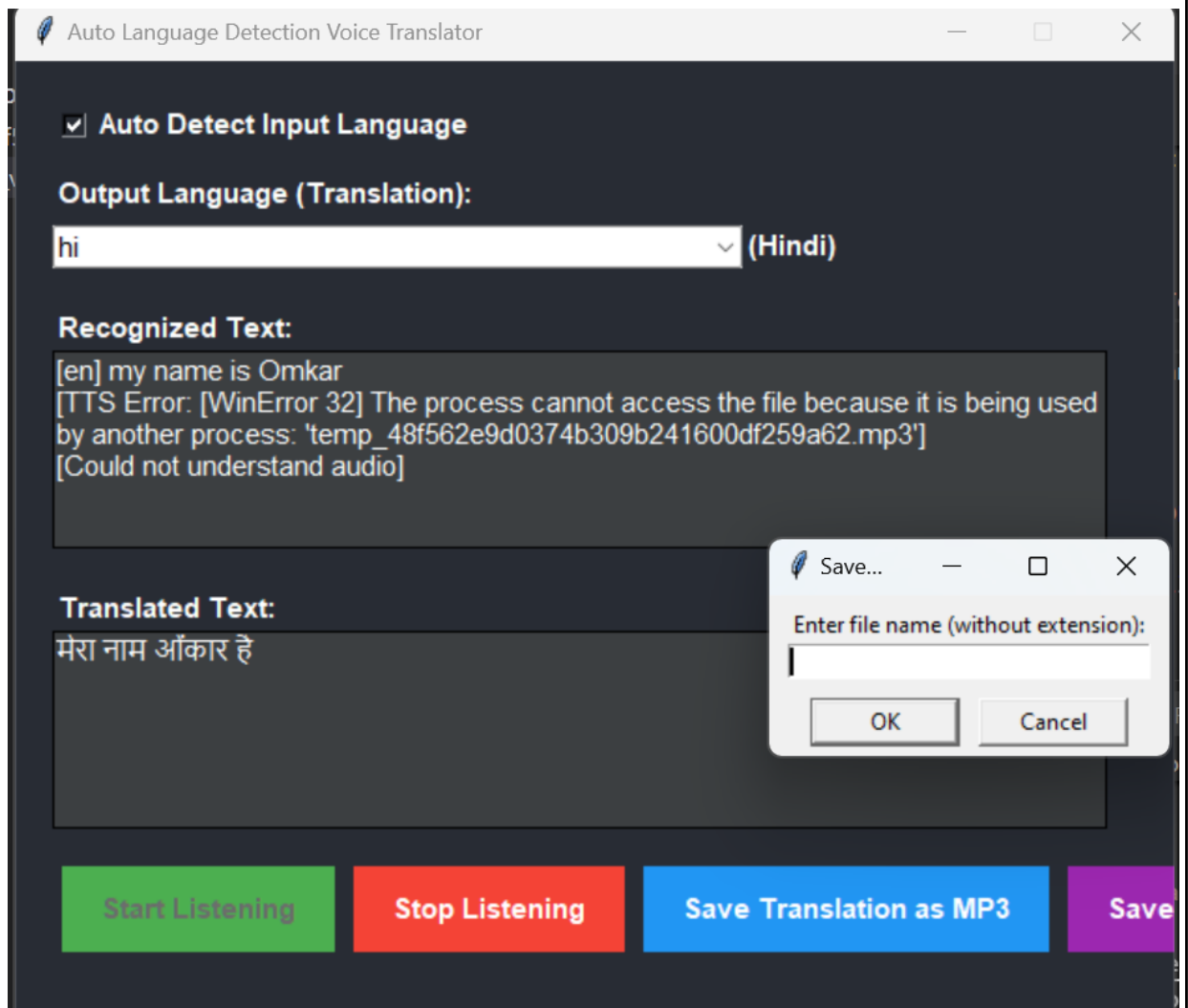
नमस्ते, मेरा नाम अकार है

Start Listening

Stop Listening

Save Translation as MP3

Save



Features of the Project

The AI-Based Real-Time Voice-to-Voice Language Translator is developed to offer an intelligent, accessible, and interactive solution for multilingual communication. The system incorporates several advanced features that enhance usability, improve accuracy, and facilitate real-time engagement. Below are the core features of the translator

1. Real-Time Speech Recognition and Translation: The application captures spoken input from the user using a microphone and immediately processes it into text using the speech recognition library. This real-time capability ensures that users can engage in live conversations without noticeable delays

2. Automatic Language Detection: To make the system more intelligent and flexible, it includes automatic detection of the input language using the googletrans library. This eliminates the need for users to manually select the source language, improving user convenience.

3. Accurate Text Translation: The system uses Google's translation API to convert the recognized input into the selected target language. This ensures context-aware and linguistically correct translations across widely spoken languages such as English, Hindi, Spanish, French, and Chinese

4. Speech Synthesis Output: Once translated, the system converts the text into audio output using the Google Text-to-Speech (gTTS) engine. The translated speech is then played using pygame, enabling complete voice-to-voice interaction.

5. Save Translations as MP3 Files: To support reusability and portability, the application allows users to save the translated audio output in MP3 format. This feature is especially useful for students, tourists, or professionals preparing preset communication prompts.

6. User-Friendly Graphical Interface: A graphical interface built with Tkinter provides users with an intuitive environment to control the translation process. Users can start and stop listening, select output languages, view both the recognized and translated text, and trigger MP3 saving with minimal effort.

7. Dynamic Threading for Real-Time Responsiveness: To maintain a responsive UI, the system uses multithreading during the listening and translation process. This ensures smooth performance and prevents freezing or delays while capturing or playing

8. Multi-Language Support: The translator supports ten languages including English, Hindi, Spanish, German, French, Arabic, Japanese, Russian, Italian, and Chinese (Simplified), making it applicable for global users

Conclusion

This project aimed to develop an AI-Based Real-Time Voice-to-Voice Language Translator to overcome the challenges of cross-language verbal communication. The application successfully integrates multiple technologies, including speech recognition, language detection, translation, and text-to-speech synthesis, into a single, interactive platform.

Using Python and widely adopted open-source libraries such as speech recognition, googletrans, gTTS, and pygame, the system is able to convert spoken input in one language into audible output in another. The graphical interface built with Tkinter enhances usability, making it accessible even to users with minimal technical knowledge.

The translator supports multiple global languages and offers real-time performance with acceptable speed and accuracy. The user can also save translated speech as an MP3 file, adding practical value for learners, travelers, and educators.

Through testing, the application demonstrated reliable accuracy in both recognition and translation tasks. The use of multithreading improved responsiveness, and the modular design provides room for future upgrades.

While the system meets its intended goals, it is not without limitations. Internet dependency, limited dialect support, and the lack of offline functionality are areas identified for improvement.

In conclusion, this translator showcases the effective application of artificial intelligence in the field of natural language processing and human-computer interaction. It is a promising step toward breaking language barriers and building more inclusive, intelligent, and accessible communication tools. With further development, it has the potential to serve a broader user base across various domains and languages.

References

- Python 3
- speech recognition
- googletrans
- gTTS
- pygame
- tkinter